MATLAB Procedure — FSAE Suspension Force Calculations

Exact commands, in order

No toolboxes required. Base MATLAB only.

PART A — Environment setup

A.1 Every script starts with these three

```matlab
clear; clc; close all;
```

Command	What it does	Why you need it
`clear`	Wipes all variables from the workspace	Stale variables from a previous run are the #1 source of "it worked yesterday"
`clc`	Clears the command window	So you can actually read the current output
`close all`	Closes all figure windows	Housekeeping

A.2 Set the display format

```matlab
format long g
```

Default `format short` shows 4 decimals. Your residual check needs to show `1.4e-12`, and `format short` will display that as `0.0000` — you'll think it's fine when it might not be. `long g` gives you 15 significant figures with automatic exponent switching.

A.3 File structure

Create one folder, five files:

File	Type	Contains
`main.m`	script	Driver — calls everything in order
`inputs.m`	script	All hardpoints and vehicle data, nothing else
`linkSolve.m`	**function**	The 8×8 solve for one corner, one load case
`skew.m`	**function**	The skew-symmetric matrix helper
`results.xlsx`	output	Written by the script

Why `linkSolve` must be a function, not a script: you'll call it 14 times (7 load cases × 2 corners). A function takes inputs and returns outputs cleanly; a script would leak variables between calls and give you cross-contaminated results.

A.4 Use code sections

Put `%%` at the start of a line to create a section break:

```matlab
%% Section 1 — Load inputs
```

Shortcut	Action
`Ctrl+Enter`	Run current section only
`Ctrl+Shift+Enter`	Run section and advance
`F5`	Run whole script
`F12`	Toggle breakpoint on current line

Running one section at a time is how you debug this in twenty minutes instead of two hours.

PART B — Data entry

B.1 Recommended: hardcode as a struct

Fastest to write, easiest to read, and you can see immediately if a number is wrong.

```matlab
F.LWB_f = [111.83, 214.50, 151.60];
```

Repeat for all 16 front points and all 16 rear points. Use the field names from the input sheet (`LWB_f`, `LWB_r`, `LBJ`, `UWB_f`, `UWB_r`, `UBJ`, `PR_wb`, `PR_rk`, `TR_o`, `TR_i`, `D_body`, `D_rk`, `WC`, `RK1`, `RK2`).

Why a struct and not a bare matrix: `F.LBJ` is self-documenting. `p(3,:)` means you have to keep a lookup table in your head, and at 1 a.m. you will index the wrong row.

B.2 Alternative: read straight from the spreadsheet

```matlab
data = readmatrix('6_0_LOTUS_SIMULATION.xlsx', 'Range', 'B15:D30');
```

Command	Use
`readmatrix(file, 'Range', 'B15:D30')`	Numeric block → matrix. What you want here.
`readtable(file)`	Mixed numeric/text → table with variable names
`readcell(file)`	Everything as a cell array
`writematrix(M, file)`	Write a matrix out
`writetable(T, file)`	Write a table out with headers

Front hardpoints are rows 15–30, columns B:D. Rear are rows 85–100.
Only use this route if you'll re-run against updated LOTUS exports. For a one-off, hardcoding is faster and less fragile.

B.3 Apply the datum shift immediately

```matlab
gz_F = F.WC(3) - RI;
```

Then subtract `gz_F` from the third column of every front point. Do this **once**, right after data entry, before anything else touches the coordinates.
To apply it across a struct, use:

Command	What it does
`fieldnames(F)`	Returns a cell array of all field names
`F.(nm)`	Dynamic field access — `nm` is a string variable

Loop over `fieldnames`, and for each one subtract the offset from element 3. Four lines.

PART C — The command reference

Every operation you need, and nothing you don't.

C.1 Vector maths

Command	Signature	Returns
`norm(v)`	vector	Scalar magnitude (2-norm)
`v/norm(v)`	—	Unit vector
`cross(a,b)`	two 3-element vectors	3-element cross product
`dot(a,b)`	two vectors	Scalar dot product
`vecnorm(M,2,2)`	matrix	Row-wise magnitudes, all at once

`cross` **orientation trap.** It accepts row or column vectors, and returns the same shape it was given. If you

write `A(4:6,i) = cross(row1, row2)` you're assigning a 1×3 into a 3×1 slot — MATLAB throws a dimension error. Force column orientation with a transpose: `cross(a(:), b(:))`.

C.2 Matrix construction

Command	Use
`zeros(8,8)`	Pre-allocate `A`
`zeros(8,1)`	Pre-allocate `b`
`eye(3)`	3×3 identity — the force-balance block for the ball joint unknowns
`A(1:3, 1:3) = eye(3)`	Assign a block
`A(4:6, 7) = something`	Assign a column slice

C.3 Solving

Command	Use	Note
`A\b`	**Solve the system**	Use this
`inv(A)*b`	Same answer, worse	Slower and numerically less stable — don't
`pinv(A)*b`	Least-squares for over/under-determined	Only for the leg decomposition in §D.8

C.4 Diagnostics

Command	Returns	Target
`norm(A*T - b)`	Residual	< 1e-6
`cond(A)`	Condition number	< 1e6
`rank(A)`	Numerical rank	Must equal 8
`det(A)`	Determinant	Non-zero (weak test — prefer `cond`)
`size(x)`	Dimensions	Check when you get a dimension error
`whos`	All variables with sizes and types	Quick workspace audit

C.5 Trigonometry — use the degree variants

Radians	Degrees	Use
`acos(x)`	`acosd(x)`	Angle between vectors
`atan2(y,x)`	`atan2d(y,x)`	Quadrant-correct angle
`sin`, `cos`	`sind`, `cosd`	Direct
—	`deg2rad(x)`, `rad2deg(x)`	Explicit conversion

Use the `d` variants throughout. Mixing radians and degrees silently is a classic and it produces answers that look plausible.

C.6 Output formatting

```matlab
fprintf('T_pushrod = %8.1f N\n', T(8));
```

Specifier	Meaning
`%8.1f`	Float, 8 chars wide, 1 decimal
`%10.3e`	Scientific, 3 decimals — use for residuals
`%s`	String
`\n`	Newline (required, `fprintf` doesn't add one)
`%%`	A literal percent sign

`disp(x)` prints a variable with no formatting. Fine for quick checks, useless for tables.

PART D — The procedure, step by step

D.1 Write `skew.m` first

The skew-symmetric matrix converts a cross product into matrix multiplication: `cross(v,F)` becomes `skew(v)*F`. You need this because in equations 4–6 the ball-joint force components are the *unknowns* — you can't cross-product an unknown, but you can multiply it by a known matrix.

Procedure:

New file, `skew.m`

First line: `function S = skew(v)`

Build a 3×3 with zeros on the diagonal and the components of `v` placed antisymmetrically. Look up the standard form — it's `[0 -v3 v2; v3 0 -v1; -v2 v1 0]` in structure, and getting the signs right is the whole point of writing it yourself.

`end`

Test it immediately before writing anything else:

```matlab
a = [1;2;3]; b = [4;5;6];
norm(cross(a,b) - skew(a)*b)
```

Must return ~1e-16. If it doesn't, your signs are transposed. **Do not proceed until this passes** — every moment equation depends on it.

D.2 Compute unit vectors

Procedure:

Define a cell array listing your six links as `{label, outboardFieldName, inboardFieldName}`

Loop with `for i = 1:6`

Inside: get the two points via dynamic field access, subtract, divide by `norm`

Store into a 6×3 matrix or a struct

Verification — do this now: print all six and compare against the table in the input data sheet. Front upper fore should read `[-0.1515 -0.9877 -0.0387]`. If your numbers match to 4 decimals, your data entry and normalisation are both correct. If they don't, stop and find the typo — everything downstream inherits it.

D.3 Build the load case table

Procedure:

Create a struct array: `LC(1).name = 'Static'; LC(1).ax = 0; LC(1).ay = 0; LC(1).nz = 1;`

Fill in all seven cases from the derivation doc

Write a loop that computes, for each case: `ΔF_z,long`, `ΔF_z,lat`, then per-corner `F_z`, then `F_x` and `F_y` via the friction circle

Useful here:

Command	Use
`numel(LC)`	Number of load cases — loop bound
`struct2table(LC)`	Convert to a table for display
`[LC.ax]`	Extract a field from all elements as a vector

Verification: after the loop, sum `F_x` across all four corners and compare to `m*ax*g`. Use `abs(sum - target) < 1e-6` and print PASS/FAIL.

D.4 Write `linkSolve.m`

Signature: takes the hardpoint struct, the contact-patch force vector, the drive moment, and which wishbone carries the pushrod. Returns the eight unknowns.

Procedure inside the function:

`A = zeros(8,8); b = zeros(8,1);`

Compute the four unit vectors you need inside: tie rod, pushrod, upper pivot axis, lower pivot axis

Rows 1–3 — force balance. The two `eye(3)` blocks go in columns 1:3 and 4:6. The tie rod unit vector goes in column 7 as a column vector. Column 8 stays zero (the pushrod doesn't act on the upright). RHS is the negative of the applied force.

Rows 4–6 — moment balance about the wheel centre. Columns 1:3 and 4:6 get `skew(position vector)`. Column 7 gets `cross(position, unitvector)`. RHS is the negative of the contact-patch moment plus the drive moment.

Row 7 — upper wishbone about its pivot axis. This is a 1×3 row: the axis unit vector times `skew(...)`, with a sign flip because the force on the *arm* is the negative of the force on the *upright*. Goes in columns 1:3 (if

the pushrod is on the lower arm) or 4:6 (if on the upper).

Row 8 — the pushrod-carrying wishbone. Same structure as row 7, plus a term in column 8 for the pushrod contribution.

`X = A\b;`

Critical for your car: the front pushrod is on the **lower** wishbone and the rear on the **upper**. So rows 7 and 8 swap roles between ends. Pass a flag argument and branch on it — don't write two nearly-identical functions, you'll fix a bug in one and forget the other.

D.5 Add the checks inside the function

Before returning, compute and store:

```matlab
out.residual = norm(A*X - b);
out.cond     = cond(A);
```

Then make it fail loudly rather than silently:

```matlab
assert(out.residual < 1e-6, 'Matrix residual too large: %.2e', out.residual);
```

`assert(condition, message, args)` throws an error and stops execution if the condition is false. This is far better than printing a warning you'll scroll past.

D.6 Build the trivial test case

Do this before you feed it real hardpoints. Twenty minutes here saves two hours of sign-hunting.

Procedure:

Copy your hardpoint struct into a test version

Replace with clean values: put the ball joints on the z axis directly above the contact patch, links along pure x and y directions, wheel centre at a round number

Apply a single pure vertical load

Solve by hand on paper — with clean geometry the answers are simple ratios

Run the function and compare

If the script reproduces your hand answer, the machinery is correct and you can trust it with real numbers. If not, you have a sign error in `skew` or in one of the moment rows, and you've found it in a case simple enough to debug.

D.7 Loop over everything

Procedure:

Nested loop: `for c = 1:2` (corners), `for k = 1:numel(LC)` (load cases)

Call `linkSolve` inside

Store into a preallocated array: `T_all(:,k,c) = X;`

Preallocate with `zeros(8, numel(LC), 2)` before the loop. Growing an array inside a loop is slow and MATLAB will underline it in orange to tell you so.

D.8 Decompose ball-joint forces into leg tensions

For the wishbone **without** the pushrod, the force lies in the leg plane, so two unknowns from three equations:

```matlab
legT = [u1(:), u2(:)] \ F_BJ(:);
```

The backslash automatically does least-squares when the system is over-determined. Check the residual — it should be near zero, which confirms the force really is coplanar (and validates equation 7).

D.9 Component calculations

Straight arithmetic, no matrix work. Use:

Quantity	Commands
Perpendicular distance to a line	`norm(cross(pointVec, dirVec))`
Angle between vectors	`acosd(dot(u1,u2))` after normalising both
Slenderness	`norm(q-p) / sqrt(I/A)`
Euler vs Johnson branch	`if slenderness > transition ... else ... end`

For the buckling branch, write it as a small function that takes the axial force and returns the critical load and λ. You'll call it for eight legs plus two pushrods.

D.10 Export

```matlab
T = array2table(data, 'VariableNames', {'Link','LC','Force_N','Type'});
writetable(T, 'results.xlsx', 'Sheet', 'LinkForces');
```

Command	Use
`array2table(M, 'VariableNames', {...})`	Matrix → table with headers
`writetable(T, file, 'Sheet', name)`	Write to a named sheet
`save('results.mat')`	Save the whole workspace
`load('results.mat')`	Restore it

Write to a separate sheet per output type. Your FEA setup then reads one file.

PART E — Debugging

E.1 Turn on error trapping before you run

```matlab
dbstop if error
```

When an error occurs, MATLAB stops **at that line** with the workspace intact, so you can inspect the variables that caused it. Without this you get a stack trace and a cleared workspace.

Turn it off with `dbclear all` when you're done.

E.2 The three errors you will hit

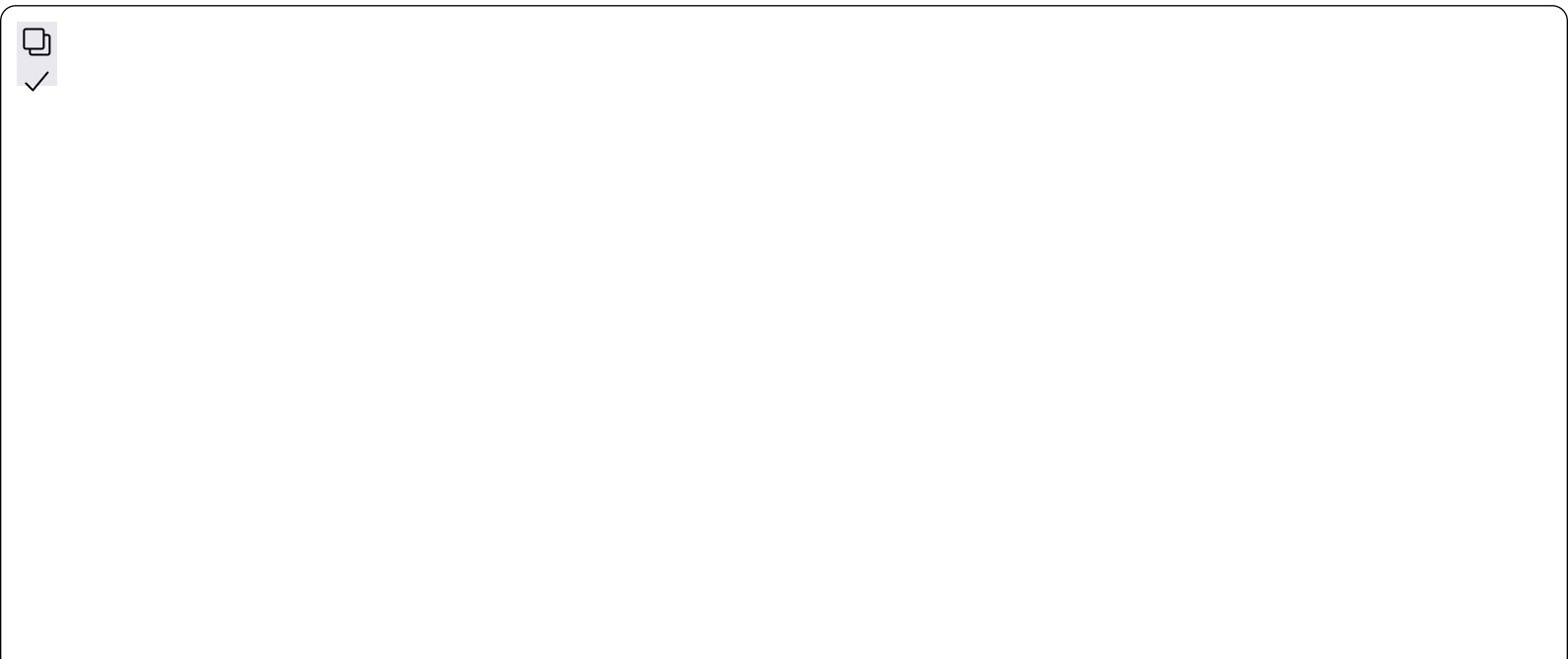
Error message	Cause	Fix
`Unable to perform assignment because the size of the left side is 3-by-1 and the size of the right side is 1-by-3`	Row/column mismatch from `cross`	Add `(:)` to force column orientation
`Matrix dimensions must agree`	Mixing a 1×3 and 3×1 in an arithmetic operation	`size()` both operands and transpose one
`Warning: Matrix is close to singular`	Duplicate or mistyped hardpoint	Check `cond(A)`, then print your hardpoints and look for two identical rows

E.3 Inspection commands

Command	Use
`size(x)`	Check orientation when you get a dimension error
`whos`	Everything in the workspace with dimensions
`disp(A)`	Print the matrix and look at it — often the fastest diagnosis
`spy(A)`	Visual plot of non-zero entries — instantly shows a block you forgot to fill
`A(4:6, :)`	Print just the moment rows

`spy(A)` is underused. Your `A` should have a very specific sparsity pattern: an identity block top-left, dense moment rows, and two sparse rows at the bottom. If it looks wrong, it is wrong.

PART F — Run order



```
1.  Write skew.m                  → test with cross() comparison
2.  Write inputs.m                → print hardpoints, eyeball against LOTUS
3.  Apply datum shift             → verify roll centre heights land at 47.01 / 81.64
4.  Compute unit vectors          → compare against the input sheet table
5.  Build load case table         → verify ΣF_x = m·a_x, ΣF_y = m·a_y, ΣF_z = W
6.  Write linkSolve.m             → test on the trivial case first
7.  Run trivial case              → must match hand calculation
8.  Run real front corner, LC5    → check ball joint ratio ≈ 1.735/0.735
9.  Loop all cases × both corners → check no link exceeds 6 kN
10. Leg decomposition             → residual near zero
11. Buckling (Euler/Johnson branch) → λ ≥ 3 on every compression member
12. Rocker forces                 → F_damper ≈ F_wheel × MR, within 10%
13. Steering cross-check          → T_TR from matrix = M_SA / 45.05
14. Export to results.xlsx
```

Do not skip steps 1, 7 and 8. They're the three places where an error is cheap to find. Every step after that inherits whatever those got wrong.

PART G — Checks to code in as assertions

Write these as `assert()` calls so the script stops rather than producing plausible-looking rubbish.

```
☐ skew(a)*b equals cross(a,b)                    tol 1e-12
☐ Unit vectors match the input sheet             tol 1e-3
☐ Σ F_z over 4 wheels = W                        tol 1e-6
☐ Σ F_x = m·a_x  and  Σ F_y = m·a_y              tol 1e-6
☐ norm(A*X - b) < 1e-6
☐ cond(A) < 1e6
☐ rank(A) == 8
☐ Pushrod force negative (compression) in bump
☐ max(abs(T)) < 6000 N
☐ Ball joint ratio: F_LBJ/F_UBJ ≈ 2.36 front, 2.20 rear    tol 15%
☐ Leg decomposition residual < 1e-6
☐ T_TR (matrix) vs M_SA/45.05                    tol 10%
☐ F_damper vs F_wheel × MR                       tol 10%
```

The last three are the ones that catch real physics errors rather than coding errors. They compare two independent routes to the same number, so agreement means your whole geometry chain is consistent.

One habit worth adopting today

After every section runs, print something and look at it. Not just the final answer — the intermediate unit vectors, the load table, the matrix condition number.

The failure mode on a deadline is a script that runs cleanly and produces numbers that are wrong by a factor of two. Nothing errors, nothing warns, and you find out in Design Event. Printing intermediate values is how you catch that, and it costs you thirty seconds per section.